

System Scanner Pro

AI-Powered Distributed Endpoint Monitoring & Remote Scanning Platform

Technical Documentation Package | v3.1

Project Documentation

Complete project reference: overview, requirements, scope, stack and team

Project	System Scanner Pro (v3.1)
Document	Project Documentation
Package date	August 21, 2026
Prepared by	Project Team - Kailasanathan (Team Leader), Saagr, Mega
Audience	Developers, administrators, testers and new team members
Status	Final

This document is part of a nine-volume documentation package generated from the actual project source code. All commands, file paths, endpoints and behaviours described here were verified against the repository at documentation time.

Table of Contents

1. Executive Summary	3
2. Introduction & Purpose of This Document	3
3. Project Overview	3
4. Problem Statement	3
5. Existing System & Drawbacks	4
6. Proposed Solution	4
7. Existing vs Proposed System	4
8. Scope of the Project	4
9. Modules Description	5
10. Architecture at a Glance	6
11. Technology Stack	6
12. Software Requirements	7
13. Hardware Requirements	7
14. Functional Requirements (FR)	7
15. Non-Functional Requirements (NFR)	8
16. Security Requirements	9
17. System Requirements Summary	9
18. Feasibility Study	9
19. Development Approach	10
20. Advantages	10
21. Limitations	10
22. Known Gaps & Not-Implemented Features	10
23. Future Enhancements	11
24. Project Timeline	11
25. Team Structure	11
26. References	11
27. Glossary	11
28. Document Revision History	12
29. Approval & Sign-off	12
30. Appendix A: Acronyms	12
31. Appendix B: Supporting Evidence	13
32. Notes on Accuracy	13

1. Executive Summary

System Scanner Pro is an **AI-powered distributed endpoint monitoring and remote scanning platform**. It consists of a central Django-based administration server and a lightweight Python client agent that runs on managed Windows, Linux or macOS machines. Agents register with the server using a registration key plus a hardware fingerprint, wait for administrator approval, and then continuously report hardware/software inventories, live health metrics, security events (USB insertion, file changes, process launches) and periodic full scans. The server presents everything through a web dashboard with real-time updates over WebSockets, statistical anomaly detection, predictive alerts (for example disk-full forecasts), asset management, license tracking, maintenance workflows and a reporting engine that exports PDF/Excel/CSV documents.

The repository at documentation time contains the admin server (Django 5), the client agent, packaging scripts, deployment assets for both a Linux VPS (nginx + systemd + Daphne) and Vercel serverless, Supabase setup SQL, automated tests and manual test plans. The project was built by a three-member intern team: Kailasanathan (team leader), Saagr and Mega.

2. Introduction & Purpose of This Document

This document is the master reference for the System Scanner Pro project. It describes what the product is, the problem it solves, the scope of the delivered build, every functional and non-functional requirement it addresses, the technology choices and the team structure. It is written so that a new developer, an evaluator or a stakeholder can understand the complete system without reading the source code first.

It is volume 01 of a nine-volume package; the remaining volumes cover system design diagrams, installation, codebase internals, module explanations, handover information, user manuals, testing/maintenance and an onboarding knowledge-transfer guide.

3. Project Overview

System Scanner Pro v3.1 is a self-hosted device-management platform. A single admin server can manage many client machines (the design has been demonstrated with dozens of simulated agents; there is no hard-coded upper limit). Each machine runs a small agent (packaged as `client_scanner.exe` on Windows) that:

- Discovers the server** - via UDP broadcast on port 45000, via a cloud registry row in Supabase (`server_registry` table), or via a direct connect URL.
- Registers** - with a unique registration key and a SHA-256 hardware fingerprint (motherboard + CPU + disk serial + MAC address), then waits in Pending status until an administrator approves it.
- Reports telemetry** - heartbeats every 30 seconds, live CPU/RAM/disk/network metrics, hardware and software inventories, and event streams from USB/file/process/software monitors.
- Executes scans** - on demand (pushed over WebSocket), on a per-device schedule, or locally triggered, producing rich JSON scan payloads stored as `ScanResult` rows and normalised into inventory tables.

Administrators use a Bootstrap-based web dashboard to approve devices, view live status, inspect scan diffs, watch anomaly/predictive alerts, manage employees, departments, locations, assets, software licenses and maintenance records, generate compliance reports, and audit every action through multiple audit-log tables.

4. Problem Statement

Small and mid-sized organisations typically track their computers, installed software, licenses and maintenance manually - spreadsheets, email approvals and ad-hoc site visits. This creates recurring problems:

- No reliable, always-current inventory of hardware and installed software.
- No early warning when disks fill up, memory saturates or machines go offline.
- No central record of who owns which asset, when warranties expire, or when licenses renew.

- Security-relevant events (USB copies, suspicious process launches, unapproved software installs) are invisible until something breaks.
- Audits and compliance checks (ISO 27001 / SOC2 / GDPR style evidence) take days of manual work.

5. Existing System & Drawbacks

Aspect	Typical existing approach	Drawback
Inventory	Spreadsheet updated by hand during audits	Instantly outdated; human error; no history
Health checks	IT staff remote-desktop into machines	Reactive; does not scale; no metrics retention
Asset ownership	Email threads and paper forms	Untraceable transfers; no assignment history
License tracking	Notes in shared drives	Missed renewals; compliance exposure
Security events	Discovered after incidents	USB/process activity not recorded centrally
Reporting	Manually compiled documents	Slow, inconsistent, hard to repeat

Table 1 - Manual approaches and their weaknesses

6. Proposed Solution

A centralised, agent-based platform where every managed machine reports to one admin server automatically:

- A tiny cross-platform Python agent collects inventories, metrics and security events and streams them to the server over HTTPS REST calls and an authenticated WebSocket.
- The Django server stores everything in PostgreSQL (Supabase) or SQLite, normalises scan payloads into queryable tables and computes health scores, anomalies and predictions.
- A role-aware web dashboard gives administrators real-time visibility and one-click actions (approve device, trigger scan, export report).
- Approval workflow ensures only trusted machines become active; re-registration of a different device under a known key demotes it back to Pending (verified by unit tests).

7. Existing vs Proposed System

Capability	Existing/manual system	System Scanner Pro
Hardware inventory	Manual spreadsheet	Automatic per-scan collection + change diffing
Software inventory	Occasional scripts	Continuous monitor + scan snapshots
Live status	Not available	30-second heartbeats + WebSocket push
Alerting	None / reactive	Rule engine + anomaly detection + predictive alerts
Approvals	Email chains	In-dashboard pending queue with audit trail
Assets/licenses	Scattered records	Full lifecycle: purchase to retirement
Compliance reports	Days of effort	One-click PDF/Excel/CSV generation
Audit trail	Usually missing	ActivityLog + AuditLog + OrgAuditLog + ComplianceLog

Table 2 - Capability comparison

8. Scope of the Project

In scope (implemented)

- Agent registration with key + fingerprint, mandatory admin approval, auto-demotion logic.
- Server discovery: UDP broadcast (45000), Supabase cloud registry, connect URLs.
- Scheduled and on-demand scanning with configurable intervals per client.

- Hardware/software/peripheral inventory with change detection between scans.
- Real-time dashboard updates over WebSockets (agent channel + dashboard channel).
- Monitoring: heartbeats, metrics history, health scoring, anomaly detection, failure-risk prediction, disk-full ETA forecasting.
- Alert engine with rules, severities, acknowledgement workflow and notifications.
- Asset lifecycle: categories, vendors, QR codes, assignments, transfers, documents.
- Org structure: locations, departments, employees, employee-asset assignments.
- Maintenance records, warranty and downtime tracking, approval workflow.
- Software license inventory with expiry tracking and assignment history.
- Intelligence module: unified alerts, ~25 report types, scheduled reports, compliance logs.
- Authentication: sessions, signed cookies, JWT pairs, hashed API keys, account lockout, audit logging of logins.
- Deployment support: VPS script (nginx/systemd/Daphne/Certbot) and Vercel serverless.

Out of scope (current build)

- Outbound e-mail delivery (password reset generates a token but sends no mail - see volume 05, section on authentication).
- Multi-factor authentication flows (model fields exist; TOTP verification not wired).
- Antivirus quarantine actions - the platform detects and alerts, it does not remediate.
- Mobile applications.
- Machine-learning model training - anomaly/predictive features are statistical (z-score, IQR, moving average, linear regression) computed on demand.

9. Modules Description

Module	Django app / package	Responsibility
Core API	admin/scanner_api	Auth, companies, clients, groups, scans, settings, API keys, org & asset models, views and URL routing
Monitoring	admin/monitoring	Device metrics, inventories, heartbeats, health scores, anomaly detection, predictions, scheduler, agent WebSocket consumer, event bus
Maintenance	admin/maintenance	Maintenance records/history/documents, warranties, downtime, licenses, compliance records, alerts
Intelligence	admin/intelligence	Unified alerts, alert rules, notifications, reports, scheduled reports, audit/compliance logs, analytics snapshots
Client agent	client/	Registration, fingerprinting, discovery, scanners, metric sampling, event monitors, WebSocket client, offline queue
Web UI	admin/templates + static	Dashboard pages built with Django templates, Bootstrap 5.3.3, Chart.js 4.4.0 and vanilla JS modules

Table 3 - Module map

10. Architecture at a Glance

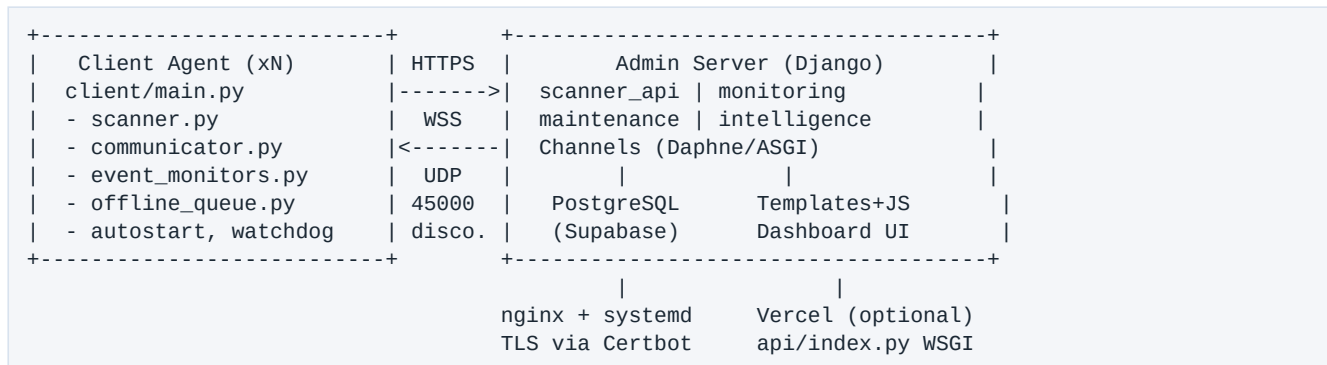


Figure 1 - High-level architecture (detailed diagrams in volume 02)

11. Technology Stack

Backend

Component	Choice	Version constraint	Where used
Language	Python	>= 3.10 required; runtime.txt pins 3.11 for Vercel	Everywhere
Web framework	Django	>=5.0, <6.0	Admin server
REST framework	djangorestframework	>=3.15, <4.0	/api/ endpoints
Realtime	channels + daphne	>=4.0 each	ASGI, WebSocket consumers
Scheduler	APScheduler	>=3.10, <4.0	Background jobs (stale checker, alert checker...)
JWT	PyJWT	>=2.8, <3.0	api/token/ endpoints
PDF engine	reportlab	>=4.0	Report engine, docs generator
Excel engine	openpyxl	>=3.1	Report exports (.xlsx)
QR codes	qrcode[pil] + Pillow	>=7.4 / >=10.0	Asset QR labels
Static serving	whitenoise	>=6.5	Vercel/serverless static files
WSGI server	gunicorn	>=22.0	api/index.py bootstrap
Database driver	psycopg2-binary	>=2.9	PostgreSQL/Supabase
DB config	dj-database-url	>=2.0	DATABASE_URL parsing
Supabase SDK	supabase	>=2.0	Cloud discovery + registration helper
CORS	django-cors-headers	>=4.0	API CORS handling

Table 4 - Backend dependencies (requirements.txt)

Client agent

- Pure Python standard library core (no third-party runtime deps except the two below).
- websockets >= 12.0 - WebSocket client connection to the server.
- watchdog >= 4.0 - filesystem change monitoring events.
- Packaged with PyInstaller 6.x into a standalone executable by build_client.py.

Frontend

- Django template language pages; no SPA framework.
- Bootstrap 5.3.3 and Bootstrap Icons 1.11.3 via CDN (dark-themed UI).
- Chart.js 4.4.0 via CDN for all graphs.

- Vanilla JavaScript ES modules under admin/static/js/ (dashboard.js, websocket.js, assets.js, executive-dashboard.js, etc.).

Data stores & infrastructure

- SQLite (admin/data/scanner.db) for local/VPS default operation, with automatic migration of legacy database files into a persistent data directory (covered by unit test).
- PostgreSQL via Supabase for cloud deployments (pooler transaction mode forced, sslmode=require, CONN_MAX_AGE=0 on serverless).
- nginx reverse proxy + systemd service + Certbot TLS on VPS; Vercel Python runtime for serverless option.

12. Software Requirements

Requirement	Detail
Operating system (server)	Windows 10/11, or Linux (Debian/Ubuntu install script provided), or macOS
Python	3.10+ local; 3.11 on Vercel (runtime.txt); verified on 3.13 dev machine
pip packages	requirements.txt (admin) and client/requirements.txt (agent)
Node/npm	Not required - frontend uses CDN assets
Database	SQLite bundled, or any PostgreSQL reachable through DATABASE_URL
Supabase account	Optional but recommended for cloud discovery and hosted Postgres
Reverse proxy	nginx config provided in deploy/nginx/scanner.conf (VPS mode)
TLS certificate	Certbot/Let's Encrypt (install.sh automates issuance)

Table 5 - Software prerequisites

13. Hardware Requirements

Tier	Minimum	Recommended
Admin server (VPS)	1 vCPU / 1 GB RAM / 10 GB SSD	2 vCPU / 2-4 GB RAM / 20 GB SSD
Admin server (local demo)	Any modern laptop/PC	8 GB RAM for browser + server comfort
Client machine	Runs alongside normal workload; agent footprint is small (pure-Python)	No special sizing needed
Network	Outbound HTTPS/WSS from clients to server; UDP 45000 on LAN for broadcast discovery	Stable connectivity; heartbeat tolerance handles brief drops via offline queue

Table 6 - Hardware sizing guidance

14. Functional Requirements (FR)

ID	Requirement	Status
FR-01	Administrator login with strong password policy (min 10 chars, mixed case, digit, symbol)	Implemented
FR-02	Account lockout after 5 failed attempts within 30 minutes; unlock window enforced	Implemented
FR-03	Signup creates company + administrator profile; first superuser auto-seeded	Implemented
FR-04	Client registers with key + hardware fingerprint; starts in Pending	Implemented
FR-05	Admin approves/rejects clients; deleted clients reactivate on approval	Implemented (unit-tested)
FR-06	Re-registration with different fingerprint resets approval	Implemented (unit-tested)
FR-07	Per-client scan interval and scan enable/disable configuration	Implemented
FR-08	On-demand scan_now command over WebSocket	Implemented
FR-09	Scan storage: JSON payload + normalised hardware/software inventory	Implemented
FR-10	Change detection between consecutive scans (added/removed/changed items)	Implemented
FR-11	Heartbeats update online/offline state; stale detector flips to offline	Implemented
FR-12	Health score 0-100 per device with component breakdown	Implemented
FR-13	Anomaly detection methods: zscore, iqr, moving_avg, threshold, compound	Implemented
FR-14	Predictive alerts: disk-full ETA (linear regression), failure risk 0-100, capacity forecast	Implemented
FR-15	Event monitors on agent: USB insert, file changes, process starts, software installs	Implemented
FR-16	Offline event queue on the agent with replay after reconnect	Implemented
FR-17	Alert rules CRUD, severity levels, acknowledgement workflow	Implemented
FR-18	Notifications centre with per-user preferences	Implemented
FR-19	~25 report types exported to PDF, Excel and CSV; scheduled report generation	Implemented
FR-20	Asset lifecycle with guarded status transitions Draft->Pending->Approved->Purchased->Assigned->Retired	Implemented
FR-21	Employee/department/location management with asset assignment links	Implemented
FR-22	Maintenance records with approval workflow, warranty and downtime logs	Implemented
FR-23	License records with expiry tracking, masked keys, assignments	Implemented
FR-24	Compliance tracking against ISO27001 / ITIL / SOC2 / internal / GDPR frameworks	Implemented
FR-25	Audit logging of logins and CRUD actions across ActivityLog/AuditLog/OrgAuditLog/ComplianceLog	Implemented
FR-26	JWT access (60 min) + refresh (7 days) token pair for API consumers	Implemented
FR-27	Hashed API keys (sk_ prefix, shown once) with rate limiting fields	Implemented
FR-28	HMAC-SHA256 signed agent requests with +/-300 s timestamp anti-replay	Implemented
FR-29	UDP discovery on port 45000 and Supabase server_registry cloud discovery	Implemented
FR-30	Password reset request/reset endpoints with signed, time-limited tokens	Partial (no email delivery)

Table 7 - Functional requirements traceability

15. Non-Functional Requirements (NFR)

ID	Category	Requirement	Evidence in build
NFR-01	Performance	Dashboard interactions feel instant; heavy work offloaded to background jobs/scheduler	APScheduler workers; JSON scan payloads indexed by client/time
NFR-02	Scalability	Support many agents without redesign	Stateless HTTP handlers + WebSocket channels; Postgres pooling via Supabase transaction pooler
NFR-03	Reliability	Agent survives network loss without losing events	Disk-backed offline queue with replay; exponential backoff with jitter
NFR-04	Maintainability	Clear app boundaries; documented code	Four focused Django apps; nine-volume documentation set
NFR-05	Portability	Run anywhere Python runs	Pure-Python agent; PyInstaller binaries for Win/Linux/macOS
NFR-06	Security	Defence in depth (see next section)	Middleware chain, HMAC, hashing, lockouts, security headers
NFR-07	Usability	Zero-training dashboard	Bootstrap dark theme, consistent layout, empty-state hints
NFR-08	Observability	Diagnosable in production	Console + file logging (local_server.log), client_crash.log, audit tables

Table 8 - Non-functional requirements

16. Security Requirements

Strong credential storage - Django PBKDF2 password hashing; passwords must satisfy the custom strength validator (10+ chars, upper/lower/digit/special).

Session integrity - HttpOnly session cookies; optional signed 'scanner_auth' cookie (TimestampSigner, salted, 30-day max age) keeps dashboards working on serverless.

Token auth - HS256 JWT pair with short-lived access tokens; secrets sourced from environment variables.

Machine identity - SHA-256 hardware fingerprints bind registrations to physical devices; mismatches demote approvals (unit-tested).

Agent transport trust - HMAC-SHA256 signatures over raw bodies plus X-Timestamp freshness window block tampering and replay.

Brute-force defence - LoginAttempt tracking with lockout; LoginHistory records every attempt with IP/user agent.

Header hardening - SecurityHeadersMiddleware sets X-Content-Type-Options, X-Frame-Options DENY, X-XSS-Protection and Referrer-Policy on every response.

Secrets hygiene rule - .env files are git-ignored and credentials must never be committed. NOTE: the shipped .env.template currently contains real-looking values and is tracked in git - flagged as a critical cleanup item in volume 06 (Handover).

17. System Requirements Summary

Combining the previous sections, a working deployment needs: one host for the admin server meeting the hardware table above; Python 3.10+/3.11 with pip; either a writable data folder (SQLite mode) or a DATABASE_URL pointing at PostgreSQL/Supabase; environment variables DJANGO_SECRET_KEY, DJANGO_DEBUG, DJANGO_ALLOWED_HOSTS and optionally SUPABASE_URL/SUPABASE_SERVICE_KEY for cloud discovery; outbound network access for agents; and for LAN discovery, UDP 45000 open between agents and server. Client machines need only OS-level Python-free execution if using the packaged binary.

18. Feasibility Study

Dimension	Assessment
Technical	Proven stack (Django + PostgreSQL + WebSockets). The entire flow - registration, approval, scanning, dashboards - is implemented and partially covered by unit tests.
Economic	Open-source components only. Runs on a low-cost VPS or free-tier serverless. No licence fees.
Operational	Single-command installer for VPS (deploy/install.sh); guided .env.template; packaged agent exe means zero Python needed on clients.
Schedule	Built incrementally by an intern team; feature-complete core delivered with tests and manuals.
Legal/compliance	Self-hosted data keeps device telemetry inside the organisation; compliance report templates support ISO27001/SOC2/GDPR evidence gathering.

Table 9 - Feasibility summary

19. Development Approach

The project follows an incremental, feature-sliced approach visible in git history: core scanning first, then monitoring/analytics, then asset/org management, then intelligence (alerts/reports/compliance), followed by deployment hardening (Supabase switch, Vercel support) and quality passes (unit tests, manual test plan, documentation). There is no formal CI pipeline beyond a scheduled Supabase registry keep-alive workflow (.github/workflows/supabase-register.yml).

20. Advantages

- Single pane of glass for every managed endpoint - inventory, health, events, assets.
- Real-time awareness via WebSockets instead of polling.
- Predictive operations: disk-full ETAs and failure-risk scores surface problems before outages.
- Strong machine identity (fingerprint + HMAC) prevents trivial spoofing of device reports.
- Compliance-ready reporting reduces audit preparation from days to minutes.
- Extremely light agent (two third-party libraries) simplifies rollout and antivirus whitelisting.
- Two deployment modes (own VPS or serverless) fit different budgets and skills.

21. Limitations

- Anomaly detection is statistical, not trained ML - unusual-but-legitimate patterns can raise false positives (tuning knobs exist per method).
- In-memory channel layer means multi-instance WebSocket fan-out requires swapping to a Redis channel layer (single-node deployments are unaffected).
- Serverless mode imposes 30-second execution caps (vercel.json maxDuration) - long scans must run on agents, which they do by design.
- Windows-centric deep metrics rely on PowerShell/CIM; Linux/macOS agents expose a smaller metric surface.
- UI ships English-only.

22. Known Gaps & Not-Implemented Features

Documented gaps (not rumours - verified in code)

These items are intentionally listed so nobody assumes they exist: (1) E-mail sending is not configured anywhere - the forgot-password flow returns a success message and stores a reset token in the Setting table but delivers no mail. (2) MFA fields exist on AdministratorProfile but no TOTP enrolment/verification flow is wired. (3) DRF DEFAULT_PERMISSION_CLASSES is left empty, so permission classes are opt-in per view rather than global. These are tracked as future work.

See volume 06 (Handover Document) for the full known-issues list including operational cleanup items such as the tracked .env.template containing real-looking credentials and the committed client_crash.log.

23. Future Enhancements

- SMTP/e-mail integration for password reset, invitations and alert digests.
- TOTP-based multi-factor authentication completing the existing model fields.
- Redis channel layer for horizontal scaling of WebSocket fan-out.
- ML-assisted anomaly baselines (per-device seasonal models) building on the existing feature-store extraction.
- Linux/macOS metric parity for the agent.
- Role-based UI restrictions surfaced more granularly (Viewer role today is read-oriented at API level).
- Automated CI running the Django test suite on push.

24. Project Timeline

Exact calendar milestones were not tracked in a formal plan document inside the repository; the following phase order is reconstructed from git history themes and is [TO BE CONFIRMED] by the team for exact dates.

Phase	Delivered content	Date
1. Foundation	Django skeleton, auth, company model, client registration/approval	[TO BE PROVIDED]
2. Scanning	Agent scanner suite, scan ingestion, scan history UI	[TO BE PROVIDED]
3. Monitoring	Metrics, heartbeats, health scores, anomaly + predictive engines, scheduler	[TO BE PROVIDED]
4. Assets & org	Employees/departments/locations/assets/licenses/maintenance	[TO BE PROVIDED]
5. Intelligence	Alerts, rules, notifications, reports engine, compliance logs	[TO BE PROVIDED]
6. Hardening	Supabase switch, Vercel path, cookie fallback, unit tests, this doc set	July-August 2026 (git evidence)

Table 10 - Development phases

25. Team Structure

Member	Role	Primary areas (evidence: MANUAL_TESTING ownership)
Kailasanathan	Team Leader / Full-stack	Authentication, dashboard, client management & scanning, client agent, WebSocket layer, API testing, integration
Saagr	Developer	Asset management, license management, compliance tracking, alert management
Mega	Developer	Intelligence & reports, notifications & audit, health scoring & monitoring

Table 11 - Team and ownership map

26. References

- Repository: <https://github.com/shinecraftinternship-kms/AdminClient.git>
- README.md - authoritative quick-start and feature list.
- MANUAL_TESTING.txt / MANUAL_TESTING.xlsx - 30 essential test cases (from 100), dated 18 July 2026.
- docs/SystemScanner_Project_Documentation.pdf + docs/generate_documentation_pdf.py - earlier single-volume documentation effort.
- setup_new_supabase.sql, setup_cloud_discovery.sql - database provisioning SQL.
- deploy/install.sh, deploy/nginx/scanner.conf, deploy/systemd/*.service - VPS deployment assets.
- Official documentation: Django (docs.djangoproject.com), Django REST Framework, Channels, APScheduler, ReportLab, Supabase.

27. Glossary

Term	Meaning
Agent / Client	The lightweight program installed on a monitored machine (client_scanner.exe).
Registration key	Unique string identifying a client enrollment; presented at registration.
Fingerprint	SHA-256 of motherboard serial + CPU id + disk serial + MAC; binds a key to physical hardware.
Heartbeat	Periodic (30 s) liveness message from agent to server.
Scan	Full inventory collection pass whose JSON result is stored and diffed.
Connect URL	Public /connect/<username>/<company_slug>/ page encoding server address + context for easy agent enrollment.
Pending	Registered-but-not-approved client state; cannot participate until approved.
Demotion	Automatic loss of approved status when identity or policy conditions fail.
Health score	Weighted 0-100 indicator combining CPU/RAM/disk/offline signals.
Failure risk	0-100 predictive score estimating near-term device failure likelihood.
Channel layer	Django Channels message bus used for WebSocket group send.
Pooler (Supabase)	Transaction-mode Postgres pooler (port 6543) suited to serverless.

Table 12 - Glossary of project terms

28. Document Revision History

Version	Date	Author	Changes
1.0	August 21, 2026	Generated from repository inspection	Initial release of the nine-volume documentation package

Table 13 - Revision history

29. Approval & Sign-off

Name	Role	Signature	Date
[TO BE PROVIDED]	Project Guide / Faculty		
Kailasanathan	Team Leader		
Saagr	Developer		
Mega	Developer		

Table 14 - Sign-off sheet

30. Appendix A: Acronyms

Acronym	Expansion	Acronym	Expansion
API	Application Programming Interface	JSON	JavaScript Object Notation
ASGI	Asynchronous Server Gateway Interface	JWT	JSON Web Token
CDN	Content Delivery Network	MFA	Multi-Factor Authentication
CRUD	Create/Read/Update/Delete	ORM	Object-Relational Mapping
CORS	Cross-Origin Resource Sharing	PBKDF2	Password-Based Key Derivation Function 2
CSRF	Cross-Site Request Forgery	RBAC	Role-Based Access Control
DFD	Data Flow Diagram	REST	REpresentational State Transfer
DRF	Django REST Framework	SQL	Structured Query Language
ERD	Entity-Relationship Diagram	SSL/TLS	Transport encryption
ETA	Estimated Time of Arrival	TOTP	Time-based One-Time Password
HMAC	Hash-based Message Authentication Code	UDP	User Datagram Protocol
HTTP/S	HyperText Transfer Protocol (Secure)	WSS	WebSocket Secure

Table 15 - Acronyms used across the package

31. Appendix B: Supporting Evidence

Every claim in this volume maps to inspected repository artefacts:

Claim area	Source files
Stack versions	requirements.txt, client/requirements.txt, runtime.txt, README.md
Auth behaviour	admin/scanner_api/{views.py, jwt_auth.py, api_key_auth.py, middleware.py}, admin/monitoring/security.py
Approval logic	admin/scanner_api/views.py + tests/test_registration_approval.py (13 assertions across scenarios)
Scheduler	admin/monitoring/scheduler.py, monitoring/management/commands/*
Analytics	admin/monitoring/{anomaly_detection.py, predictive.py, health.py, feature_store.py}
Reports	admin/intelligence/report_engine.py, models.py (REPORT_TYPE_CHOICES)
Agent behaviour	client/main.py, communicator.py, discovery.py, scanner.py, metrics.py, events/*, key_manager.py, fingerprint.py, runtime.py
Deployment	vercel.json, api/index.py, deploy/install.sh, deploy/nginx/scanner.conf, deploy/systemd/
Testing	admin/scanner_api/tests/, MANUAL_TESTING.txt/.xlsx

Table 16 - Evidence index

32. Notes on Accuracy

Verification statement

This document was produced by inspecting the actual source tree on August 21, 2026. Version strings differ slightly across components (README says v3.1, the admin banner shows v2.1, the agent prints v1.6.1, MANUAL_TESTING references v3.0); where they conflict, this package treats the README version (v3.1) as canonical and notes discrepancies in volume 06. Nothing in this package is invented: unknown values are marked [TO BE PROVIDED], unimplemented features are labelled NOT IMPLEMENTED, and unexecuted tests are labelled TO BE TESTED.